# Self-Evolution Mechanism

# Hermes Agent Self-Evolution Mechanism

## Table of Contents



---

## 1. Project Overview

**Hermes Agent** is a self-improving AI agent developed by Nous Research, positioned as a "general-purpose, self-evolving, cross-platform" AI assistant. Its core philosophy is:

> *"Create skills that learn from experience, continuously improve through use, and run anywhere."*

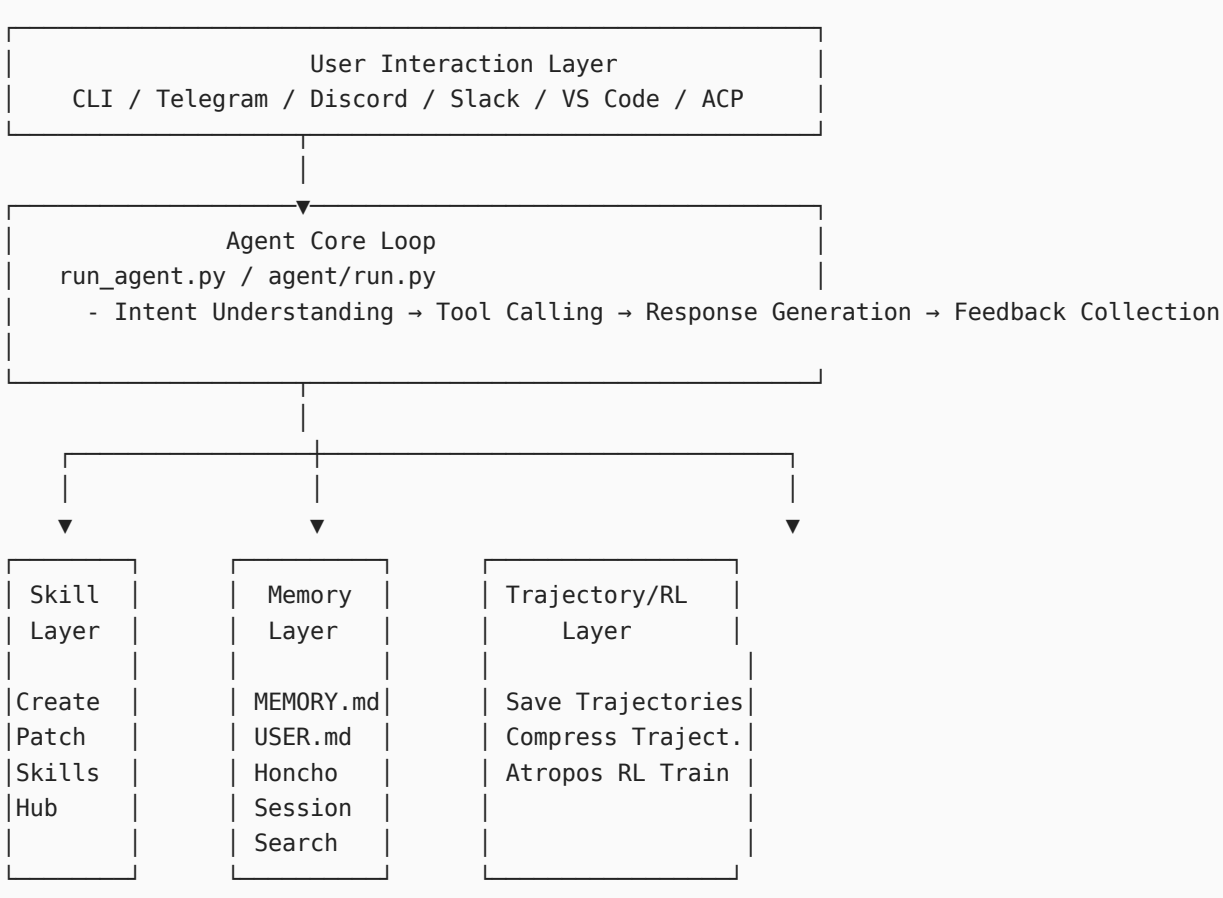
### Key Features

| Feature | Description |
|---|---|
| Built-in Learning Loop | Discover and solidify knowledge from complex tasks, error corrections, and non-trivial workflows |
| Memory System | File-based persistent memory (MEMORY.md / USER.md) + Honcho user modeling |
| Cross-Session Retrieval | FTS5 full-text search + LLM summarization compression, supporting context reuse across multiple sessions |
| Skill Management | Agent can autonomously create, edit, and patch skills, self-maintaining at runtime |
| RL Training Support | Trajectory saving, compression, and integration with Tinker-Atropos RL environment |
| Multi-Platform Gateway | Telegram, Discord, Slack, WhatsApp, Signal and other messaging platforms |
| MCP Integration | Dynamic discovery and invocation of external tools provided by MCP servers |

---

## 2. Self-Evolution Architecture Overview

Hermes Agent's self-evolution is not achieved by modifying its own code, but through a **multi-layer persistent knowledge management system** that forms a closed loop:




### Self-Evolution Feedback Loop

```
Execute Task
  ↓
Detect Complex Task / Error / New Workflow
  ↓
skill_manage Creates or Patches Skill ──→ Skill Persisted to ~/.hermes/skills/
  ↓
```

```
Update Memory Files (MEMORY.md / USER.md)
  ↓
Save Trajectories (batch_runner.py / trajectory.py)
  ↓
RL Training Data Compression (trajectory_compressor.py)
  ↓
Next Session: Retrieve Historical Skills / Memory / Sessions → Reuse Experience
```

> **Note**
>
> **Feedback signals are the key to the self-evolution closed loop:**
>
> - Agent's self-evolution is essentially a "perceive-feedback-adjust" closed-loop process. Whether it's skill creation, memory updates, or trajectory compression, all depend on clear feedback signals to drive: Was the task successful? Is the user satisfied? How does it compare to historical solutions?
> - This is highly consistent with the design philosophy of Reinforcement Learning (RL): the external environment provides rewards/penalties, and the agent adjusts its strategy accordingly. The difference is that the Agent's feedback signal sources are more diverse, coming not only from task success rates but also from multi-dimensional information such as user explicit corrections, tool call exceptions, and context compression loss assessments.

---

# 3. Core Self-Evolution Mechanisms in Detail

## 3.1 Skill Creation and Self-Maintenance

### 3.1.1 Skill Management Tool

**File**: `tools/skill_manager_tool.py`

The Agent performs complete CRUD (Create, Read, Update, Delete) operations on skills through the `skill_manage` tool:

```
# Supported action parameters
CREATE   # Create new skill (extracted from successful experience)
EDIT     # Edit complete content of existing skill
PATCH    # Small-scale patch (quick fix for errors/omissions)
DELETE   # Delete obsolete skills
WRITE_FILE   # Create skill sub-files (references, templates, assets)
REMOVE_FILE  # Delete skill sub-files
```

### 3.1.2 Feedback Signals and Trigger Conditions (Design Philosophy)

> **Note**
>
> **Core Design Insight: Implicit Signal Dependency is a Double-Edged Sword**
>
> Hermes Agent's skill triggering mechanism is built on a **hybrid feedback signal system**, and its core contradiction is: **The most valuable feedback signals (whether a task is valuable, whether it's worth crystallizing) are precisely the hardest to quantify.**

**Trigger Conditions for Creating Skills:**

| Trigger Scenario | Example |
|---|---|
| **Complex task (5+ tool calls) completed successfully** | Completed a refactoring task spanning multiple files |
| **Overcame a tricky error** | Discovered and worked around a non-obvious bug |
| **Method worked after user correction** | Successfully completed task after user pointed out correct direction |
| **Discovered a non-trivial workflow** | Found a better CI/CD pipeline |

**Distribution of Quantitative vs. Qualitative Signals:**

| Trigger Condition | Signal Type | Verifiability | Advantage | Risk |
|---|---|---|---|---|
| Complex task (**5+ tool calls**) | Explicit quantitative | High (precise counting) | Reproducible, unambiguous | Cannot distinguish "complex useless work" from "complex but valuable" |
| **Tricky error** overcome | Implicit semantic | Low (depends on LLM judgment) | Can capture truly valuable experience | May over-create skills (create for every error) |
| **Non-trivial workflow** discovered | Implicit semantic | Low | Can capture unpredictable patterns | Judgment is subjective, depends on model's own metacognition |
| **After user correction** method worked | Implicit social | Medium (requires understanding correction intent) | User participation reduces error cost | Over-dependence on user intervention, weakening autonomy |

> **Important**
>
> **Insight: CREATE trigger is essentially "Experience Distillation"**
>
> - The Agent not only executes tasks but continuously evaluates during execution "whether this experience **can be generalized**" -> because it's self-evolving, there may also be an "experience overfitting" problem
> - The 5+ call threshold reduces misjudgment cost: if a workflow repeats multiple times and each time uses 5+ tools, it's likely worth solidifying
> - This is essentially an **exploration-exploitation trade-off** in Online Learning: the Agent decides while executing whether to record the current experience into long-term memory

### 3.1.3 Maintenance Trigger Conditions

The Agent is explicitly instructed to **automatically patch skills** in the following situations:

| Trigger Scenario | Maintenance Action |
|---|---|
| Encountered problem when using skill | Immediately PATCH, don't wait to be asked |
| Instructions are outdated or incomplete | Complete missing steps and known pitfalls |
| OS-specific failures | Add platform-related patches |
| Discovered new pitfalls | Add warnings in SKILL.md |

### 3.1.4 Skill File Format

Skills use a **YAML frontmatter + Markdown body** structured format:

```
---
name: git-interactive-rebase
```

```
description: Use interactive rebase to organize commit history
version: 1.2.0
platforms: [macos, linux]
tags: [git, vcs, history]
author: hermes-agent  # marked when agent creates itself
auto_updated: true    # marked as agent self-maintained
---

# Git Interactive Rebase Skill

## Applicable Scenarios
Use when you need to organize commit history, merge WIP commits, or modify commit messages.

## Usage Steps
1. Use `git log --oneline -n 20` to view recent commits
2. Determine the number n of commits to modify
3. Execute `git rebase -i HEAD~n`
4. Select operations in the editor (pick/reword/squash/fixup/drop)

## Common Pitfalls
- ⚠️ Do not rebase commits that have been pushed (unless sure there's no collaboration)
- ⚠️ When handling conflicts, resolve them in order, don't skip
```

> **Note**
>
> Anthropic's proposed progressive disclosure and lazy loading workflow

### 3.1.5 Skill Directory Structure

```
~/.hermes/skills/
├── git-interactive-rebase/
│   ├── SKILL.md              # Main skill file
│   ├── references/           # Reference documents
│   │   └── git-rebase-demo.md
│   └── templates/            # Output templates
│       └── commit-message-template.txt
├── docker-debugging/
│   └── SKILL.md
└── swe-expert/
    └── SKILL.md
```

### 3.1.6 Skill Registration to System Prompt

**File**: `agent/prompt_builder.py`

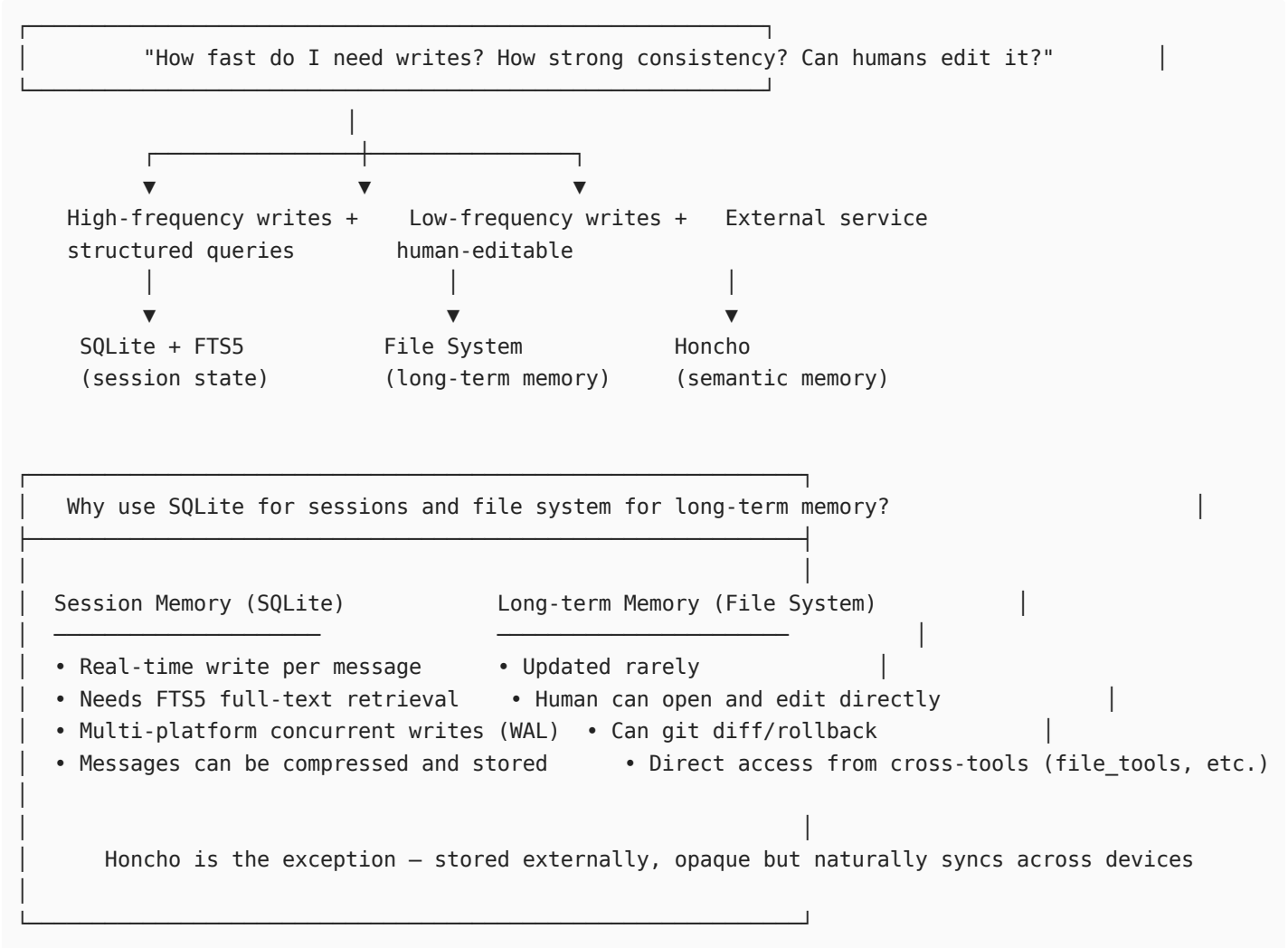
Skills are **automatically loaded** into the Agent's system prompt through `build_system_prompt()` in `prompt_builder.py`:

```
def build_system_prompt(identity, hints, skills, env_vars, soyl):
    # 1. Load identity description (SOUL.md)
    # 2. Apply hints — includes skill usage guidance
    # 3. Inject active skills list and their descriptions
    # 4. Expose environment variables
    # 5. Output complete system prompt
```

## 3.2 Memory Persistence System

Hermes Agent constructs a three-layer memory system to achieve cross-session knowledge persistence.

### 3.2.0 Storage Medium Layered Design




**Three-Layer Memory Storage Medium Comparison:**

| Memory Layer | Storage Medium | Write Frequency | Query Mode | Human Editable | Version Control |
|---|---|---|---|---|---|
| Session State | SQLite + FTS5 | Per-message real-time write | Structured + full-text search | ❌ | ❌ |
| MEMORY/USER.md | File System (`~/.hermes/memories/`) | Very low (on-demand) | Full-text read | ✅ Direct edit | ✅ Git |
| Honcho | External Service (HTTP API) | On-demand | Semantic search | ❌ | ❌ |

> **Note**
>
> **Storage medium selection is the result of trading off "access frequency × consistency requirements × human editability".**
>
> These three dimensions form the "Impossible Triangle" of storage selection:

- **High-frequency writes + strong consistency + human-editable** → Currently no single storage medium can satisfy all three
- Therefore, Hermes Agent anchors the three memory layers to different trade-off poles instead of pursuing a unified solution

**Design Insight 1: Write frequency determines storage topology**

| Dimension | Session State (SQLite) | Long-term Memory (File System) | Honcho (External API) |
|---|---|---|---|
| Write frequency | **Per-message real-time** (millisecond level) | **Very low (on-demand)** (monthly level) | **On-demand sync** (non-real-time) |
| Access pattern | Passive (waiting for query trigger) | Proactive (system prompt load) | Passive (user profile query) |

The core reason for choosing SQLite at the session layer is not "speed", but "**can handle high-frequency random write concurrent pressure**": when each message is persisted, it needs WAL concurrent control, FTS5 index updates, token counting — operations that file systems cannot efficiently complete in multi-client scenarios. Conversely, long-term memory has extremely low write frequency, and the designer's judgment is "it's better to read slightly slower each time (load entire file) but ensure humans can edit directly without additional tools".

**Design Insight 2: Consistency requirements determine synchronization strategy**

The three memory layers have natural **consistency level differences**:

- **SQLite**: ACID transactions + WAL, intra-process consistency guaranteed, but cross-process requires locks
- **File System**: Depends on OS page cache, no transaction semantics, but naturally fits Git's version control model
- **Honcho**: Eventual consistency (HTTP API), server is the source of truth, local cache may temporarily lag

This forms a **consistency descending chain**: `SQLite > File System > Honcho`. The designer's bet is: the higher the access frequency, the higher the consistency requirement; the lower the access frequency, the greater the latency tolerance allowed.

**Design Insight 3: Human editability is the externalization of "trust"**

The file system as the carrier for MEMORY.md / USER.md is not just because it's "convenient", but because it conveys a design philosophy: **The Agent's knowledge is not the Agent's private property, but something humans can review, intervene in, and even override.**

This contrasts sharply with the black-box nature of RL training data:

- Training data after trajectory compression: cannot be directly inspected by humans (requires decoding ShareGPT JSONL)
- MEMORY.md: can be read, modified, and rolled back at any time

It can be said that the file system layer is Hermes Agent's "trust anchor" to humans — no matter what the Agent learns and remembers, humans always retain final editing rights. This is an **institutional design to prevent capability runaway**.

**Design Insight 4: Version control is an extension of "human editability"**

Putting MEMORY.md into Git's version control system means memory has:

- **Audit capability** (who changed what at what time)
- **Rollback capability** (Agent got dumber? One-click restore to previous version)
- **Branch experiments** (can open a branch to test different memory strategies)

This is an advantage the SQLite session layer doesn't have — you can't `git diff` sessions.db. The designer apparently believes: **Long-term knowledge is worth version control, short-term state is not**.

**Design Insight 5: Honcho's "exception" reveals platform-level design trade-offs**

Honcho is stored in an external service rather than locally, which is a **deliberate cross-device synchronization design**. The cost is:

- Not available offline
- Data is opaque (can't see raw storage)
- Consistency depends on network and third-party services

The designer accepted these costs in exchange for: **Users can get consistent user profiles on any device**. This is a typical "C.A.P. trade-off" practice — between consistency and availability, Honcho chooses availability (weak consistency + cross-device sharing).

**Important**

**Summary: Three-layer memory is an "information lifecycle" management system**

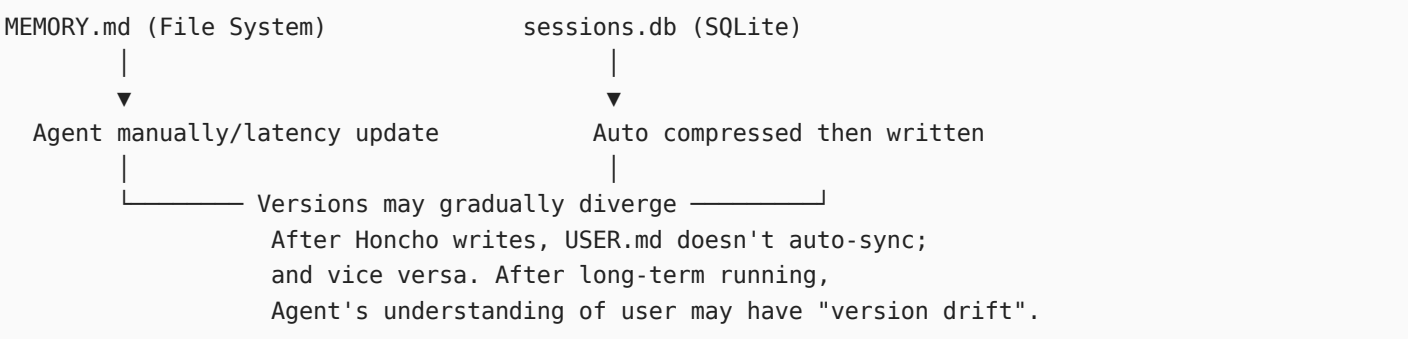
```
Storage medium selection = argmin(access frequency mismatch cost + consistency mismatch cost + editability mismatch cost)
```

The three layers correspond to three lifecycle stages of information:

- **Session Layer (SQLite)**: When information is "hot" → High-frequency writes, fast retrieval
- **Long-term Memory Layer (File System)**: When information is "warm" → Human intervention, low-frequency updates, traceable
- **Semantic Memory Layer (Honcho)**: When information is "cold" → Cross-device sharing, eventual consistency, model distillation

No single layer is the "best" storage, but together they exactly cover the full lifecycle needs of Agent memory.

**Cross-layer consistency potential risks (not covered by current framework):**




### 3.2.1 First Layer: File Memory (MEMORY.md / USER.md)

**File**: `tools/memory_tool.py`

**MEMORY.md** — Agent's private notes:

- Environmental facts (project structure, tool habits, system differences)
- Project conventions (code style, naming conventions)
- Tool defects and known issues

- Configuration information (key configs that avoid detours)

**USER.md** — Agent's understanding of the user:

- User's role and responsibilities
- Preferences and communication style
- Expectations and need patterns
- Collaboration habits

### 3.2.2 Second Layer: Session State (SQLite + FTS5)

**File**: `hermes_state.py`

```
# SQLite Schema
CREATE TABLE sessions (
    id TEXT PRIMARY KEY,         # Unique session identifier (UUID)
    source TEXT,                 # 'cli' | 'telegram' | 'discord' etc., supports cross-platform
retrieval
    model TEXT,                  # Model name used in this session (e.g., 'claude-sonnet-4-6')
    created_at INTEGER,           # Unix timestamp (seconds), session creation time
    updated_at INTEGER,           # Unix timestamp (seconds), most recent message time, used for
LRU cleanup
)

CREATE VIRTUAL TABLE sessions_fts USING fts5(
    # ── FTS5 Full-Text Index Columns (content actually stored in sessions table) ──────────
    content,                     # Message body (version after context compression)
    role,                        # Message role: 'user' | 'assistant' | 'tool'
    session_id,                  # Foreign key to sessions.id, supports grouping retrieval by
session
    token_count,                 # Token count for this message, used for sorting and pagination
    # ── FTS5 Content Sync Config ──────────────────────────────────────────────────────────
    content='sessions',          # Points to sessions table (not independent storage), avoids data
redundancy
    content_rowid='rowid',       # Associates with original records via rowid, ensures index and
table always consistent
);
```

**Features:**

- WAL mode: concurrent reads + single write
- FTS5 full-text index: millisecond-level cross-session search
- Distinguish session sources by `source` tag
- Support message-level token counting

### 3.2.3 Third Layer: Skill System (Persistent Knowledge Modules)

**File**: `tools/skills_tool.py` / `tools/skill_manager_tool.py`

Skills as **executable, patchable persistent knowledge modules**, going further than MEMORY.md:

- Have structured metadata (name, version, platform, tags)
- Can contain sub-files (references, templates, assets)
- Support version control (explicit version field)
- Have active/inactive states (can be toggled in config)

- Have security scanning mechanism (validate before installation)

> **Note**
>
> **Skills vs. Memory: When to沉淀 to skills, when to沉淀 to memory?**
>
> The schema description in `memory_tool.py` gives the most direct guidance:
>
> ```
> "Do NOT save task progress, session outcomes, completed-work logs, or temporary TODO
>  state to memory; use session_search to recall those from past transcripts."
> "If you've discovered a new way to do something, solved a problem that could be
>  necessary later, save it as a skill with the skill tool."
> ```
>
> **Core Distinction: Declarative Knowledge vs. Procedural Knowledge**
>
| Dimension | MEMORY.md / USER.md | Skills |
|---|---|---|
| **Knowledge Type** | Declarative (what it is) | Procedural (how to do it) |
| **Typical Content** | "This project uses Poetry instead of pip" | "How to add a new dependency for this project: poetry add xxx" |
| **Update Frequency** | Very low (cross-month) | Low (weekly level, but patches are frequent) |
| **Structuredness** | Unstructured entries (natural language) | YAML frontmatter + Markdown structure |
| **Capacity Limit** | 2200/1375 characters (strict limit) | Unlimited (can include sub-files) |
| **Reuse Pattern** | Passive context hint | Proactive tool call trigger |
| **Executable** | ❌ Read-only | ✅ Includes steps, commands, verification methods |
>
> **When to choose Skills (skill_manage):**
>
> 1. **Multi-step workflows**: Tasks requiring 5+ tool calls to complete
> 2. **Reproducible success path**: Discovered a standard path to solve a certain type of problem
> 3. **Needs precise instructions**: Includes specific commands, parameters, verification steps
> 4. **Has pitfalls to warn about**: Common traps to avoid in this type of task
> 5. **Cross-platform differences**: Execution approach needs adjustment based on OS
>
> Example scenarios:
>
> - "How to add a new microservice in this monorepo"
> - "How to write integration tests for this project"
> - "How to debug timeout issues in this CI pipeline"
>
> **When to choose Memory:**
>
> 1. **Static facts**: Environment configuration, tool versions, conventions
> 2. **User preferences**: Communication style, tech stack preferences, pitfalls encountered
> 3. **One-time discoveries**: Useful next time but not worth solidifying a complete workflow
> 4. **Configuration info**: API endpoints, special environment variables
>
> Example scenarios:
>
> - "This user prefers to communicate in Chinese"
> - "This project's Python version is 3.11"
> - "User doesn't like being interrupted directly"

**A Decision Tree:**

```
Is this knowledge "how to do a certain type of task"?
        │
        ├── YES → 沉淀 to Skills (skill_manage)
        │         - Needs 5+ steps?
        │         - Needs precise commands and parameters?
        │         - Has pitfalls and verification steps?
        │         → Yes → Skills
        │
        └── NO → Is this knowledge "about who/what"?
                  │
                  ├── YES → 沉淀 to Memory (memory)
                  │         - User preferences → USER.md
                  │         - Environmental facts → MEMORY.md
                  │
                  └── NO → Does this knowledge need cross-session retrieval?
                            └── YES → Hand to session_search (FTS5)
```

**Deep interpretation of design philosophy**

1. **Memory is "lightweight cache", Skills are "heavyweight index"**
   - Memory entries are easy to write, character-limited, implying "don't over-remember"
   - Skills have versions, platforms, security scans, implying "skills need maintenance cost"
2. **Skill existence itself is an assertion about task complexity**
   - If you decide to create a skill, you're saying "this problem is worth standardizing"
   - Skill creation threshold (needs to write SKILL.md) is much higher than memory (one add call)
3. **Complementarity of memory and skills**
   - Skills can reference content in memory (e.g., "see project conventions in MEMORY.md")
   - Memory can point to skills (e.g., "deployment process for this project is in deploy-workflow skill")
   - The two are not substitutes but complementary

### 3.2.4 Feedback Signal Design Philosophy

**Important**

**Core Design Insight: The feedback signal for memory is essentially "expected information lifespan"**

Hermes Agent's memory system is not passive storage, but an **active information lifecycle management system**. Different memory layers correspond to different feedback loop frequencies and information fidelity requirements.

**Three-Layer Memory Signal Feedback Characteristics Comparison:**

| Memory Layer | Feedback Signal Source | Update Frequency | Signal Type | Consistency Mechanism |
|---|---|---|---|---|
| MEMORY.md | Agent self-assessment (LLM judges "is this worth remembering") | Very low (cross-month) | Implicit semantic | Frozen snapshot mode |
| SQLite Session | External retrieval query triggers | Medium (weekly level) | Explicit query + FTS5 | WAL concurrent control |

| Memory Layer | Feedback Signal Source | Update Frequency | Signal Type | Consistency Mechanism |
|---|---|---|---|---|
| Skill System | Skill execution results + user feedback | Low (weekly level) | Implicit + explicit mixed | Version field + snapshot cache |

**Design Philosophy:**

| Design Decision | Reason |
|---|---|
| Frozen Snapshot Mode | Stable system prompt, but tool responses show latest state |
| Character Limit (not token limit) | Model-agnostic (doesn't depend on specific model's tokenizer) |
| Injection attack scanning | Prevent user-injected malicious content from manipulating Agent |

**Design Philosophy of "Frozen Snapshot" Mode (Frozen Snapshot)**

- System prompt remains stable during session, but each `memory` tool response reflects the latest state
- This solves the **consistency-freshness contradiction** in LLM inference: inference needs stable context, but reality is continuously changing
- Insight: This is actually a kind of **optimistic cache + delayed invalidation** strategy — assuming the snapshot is good enough in the short term, using tool calls to get incremental updates
- Contrast: Another approach would be to reload all memory each time, but this would cause unnecessary context overhead for each inference

**Deep Reason for Character Limit (not token limit)**

- Tokenization is model-bound, different models have different tokenizers
- But the deeper design intent is: **Let memory content be model-independent, becoming true "persistent knowledge"**
- This allows memory to migrate between different models, and makes memory management tools more stable

**Reverse Signal Value of Injection Attack Scanning**

- The scan itself is also a kind of "negative feedback" signal: any blocked content is a manipulation signal users tried to inject
- This passive detection signal can in turn help identify **high-risk user interaction patterns** (is someone continuously trying to inject?)

---

## 3.3 Cross-Session Retrieval and Pattern Recognition

**File**: `tools/session_search_tool.py`

This is the key mechanism for Hermes Agent to achieve "experience reuse".

### Workflow

```
User queries "How to handle concurrency in Python?"
       ↓
FTS5 Full-text Search → Find related sessions (top-3 unique sessions)
       ↓
Group by session, get related messages in session
       ↓
LLM Summary (cheap fast model)
       ↓
Return: Focused summary (not original transcript)
```

```
        ↓
Agent integrates historical experience into current task
```

### Synergy with Memory System

| Search Target | Tool | Storage Location |
|---|---|---|
| Historical dialogue experience | `session_search` | SQLite FTS5 |
| Agent's own notes | `memory` | MEMORY.md |
| User preferences | `memory` + `honcho` | USER.md + Honcho |
| Structured skills | `skills` / `skill_manage` | ~/.hermes/skills/ |

---

## 3.4 Context Compression and Information Protection

**File**: `agent/context_compressor.py`

### Compression Strategy

```
# Protected areas (never compress):
PROTECTED_HEAD = ["system", "first_human", "first_gpt", "first_tool"]

# Compression area:
# - Middle messages exceeding token_budget
# - Compressed by LLM summary
# - Retain enough context for tool calling

# Tail protection:
# - Last ~20K tokens remain original (latest context is most important)
```

### 3.4.1 Feedback Signal Design for Context Compression

> **Important**
>
> **Core Design Insight: The feedback signal for compression is "information compressibility" not "information importance"**
>
> The core assumption of context compression is: **After summary compression, historical information from middle rounds can still retain enough "signals" to drive correct inference.** This is an empirical assumption, and context compression in current Agent products typically follows this paradigm

**Signal Value Analysis of Three-Layer Protection Mechanism:**

| Protection Layer | Protected Content | Signal Source | Design Logic |
|---|---|---|---|
| **Head Protection** | system + first exchange | Structural position | Inference framework foundation, entire session becomes meaningless once destroyed |
| **Tail Protection** | Recent ~20K tokens | Dynamic token budget | Latest context directly affects current inference quality |
| **Middle Compression** | Historical interactions | LLM summary quality | "Semantic summary" of historical experience retains patterns rather than details |

**Design Value of Iterative Summary (`self._previous_summary`) -> Segmented Progressive Incremental Compression Method**

- First compression: Generate summary from scratch
- Second+ compression: Update based on previous summary, rather than re-summarizing
- This is consistent with **differential compression** thinking: incremental changes only need to record incremental information
- Avoids signal degradation from over-summarization of session history ("summary of summary of summary") after multiple compressions

**Design Intent of Structured Summary Template (Goal/Progress/Decisions/Files/Next Steps)**

- References experience from **Pi-mono and OpenCode**
- Each field corresponds to an information type: goal (direction), constraints (preferences), progress (state), decisions (path), files (resources), next steps (plan)
- Insight: This six-tuple actually encodes the complete **MDP state** of agent task execution (MDP = Markov Decision Process)

```
Message list
    ↓
Identify PROTECTED_HEAD (system-level messages)
    ↓
Identify PROTECTED_TAIL (recent ~20K tokens)
    ↓
Middle region → LLM Summary → Single summary message replacement
    ↓
Verify compressed token_count ≤ token_budget
    ↓
If still over limit, recursive compression (re-summarize)
```

---

## 3.5 RL Training Data Pipeline

Hermes Agent has a built-in complete RL training data pipeline, **capable of converting Agent experience into trainable RL data**.

**Core Design Insight: Hermes Agent's RL pipeline is a "Verification-Data-Training" three-layer decoupled architecture**, **Signal flow in three-layer decoupled architecture:**

```
Layer 1: Trajectory Collection (HermesAgentLoop)
  → Output: AgentResult(messages, turns_used, tool_errors, reasoning)
  → Feedback signal: Tool call sequences + error records (structured intermediate process data)

Layer 2: Reward Computation (subclass compute_reward + ToolContext)
  → Input: AgentResult + ToolContext (complete tool access permissions)
  → Output: float reward (usually 0.0-1.0, but any float is valid)
  → Feedback signal: Defined by subclass (reward function can be anything: code correctness, user
satisfaction, or custom metrics) — this is the most critical signal source

Layer 3: Training Data Compression (TrajectoryCompressor)
  → Input: Raw trajectory + reward
  → Output: Compressed trajectory (ShareGPT format)
```

```
  → Compliance constraint: Training framework context window upper limit (15,250 tokens) —
exceeding this won't fit into trainer, this is a hard requirement
```

> **Note**
>
> **Note: Layer 3 does not use ContextCompressor**
>
> Trajectory compression is not "context compression within sessions"; they are two completely independent modules:
>
> - `agent/context_compressor.py`: Triggered in real-time during session, operates on `{role/content}` format, serving **inference**
> - `trajectory_compressor.py`: Offline batch processing after session ends, operates on **ShareGPT JSONL**, serving **training data generation**
>
> The only commonality is both call `call_llm()` for LLM summarization, but even the prompt templates are different — **the former outputs a structured six-tuple, the latter outputs natural language description**.

---

**ToolContext as the "Universal Interface" for Reward Functions**

- `compute_reward()` receives complete tool access permissions (terminal, file, web, browser...)
- This means reward functions can **directly execute real-world verification**: not only see what the Agent said, but verify it did it correctly
- Example: Check if files were actually created, commands actually succeeded, tests actually passed
- Insight: This is essentially **Ground Truth verification**, not LLM self-assessment — the reward signal comes from the objective world, not the Agent's self-perception

> **Note**
>
> The innovation of the RL pipeline is the **complete decoupling of reward function and training framework**: `compute_reward()` is an abstract method implemented by subclasses, meaning **reward functions can be anything: code correctness, user satisfaction, or custom metrics**.

---

**Signal Quality Differences in Two-Phase (Phase 1 / Phase 2) Design**

| Phase | Server Type | Token Tracking | Applicable Scenario | Training Signal Quality |
|---|---|---|---|---|
| Phase 1 | OpenAI Compatible | None (placeholder tokens) | SFT data generation, verification tests | Cannot be used for RL (missing logprobs) |
| Phase 2 | VLLM/SGLang ManagedServer | Exact token IDs + logprobs | RL training (GRPO/PPO) | Complete (can be used for advantage estimation) |

- Phase 1 trajectory data has messages but no exact token-level signals, cannot compute advantage estimation
- Phase 2 is true RL training mode, but requires VLLM server infrastructure
- This phased design allows **first collect data (Phase 1), then upgrade to training (Phase 2)**, reducing cold-start threshold

### 3.5.1 Trajectory Saving

**File**: `agent/trajectory.py` / `trajectory_compressor.py`

Trajectory is the complete sequence of Agent interacting with the environment:

```
{
  "session_id": "xxx",
  "model": "claude-sonnet-4-6",
  "trajectory": [
    {"role": "user", "content": "Help me refactor this module..."},
    {"role": "assistant", "content": "...", "tool_calls": [...]},
    {"role": "tool", "tool_call_id": "...", "content": "..."},
    {"role": "assistant", "content": "Okay, I'll start refactoring..."},
  ],
  "success": true,
  "tool_count": 7,
  "created_at": 1712000000
}
```


### 3.5.2 Trajectory Compression — "Lossy Compression" of Training Signals

> **Important**
>
> **Core Design Insight: For training trajectory compression, quality depends on understanding "where are the training signals"**
>
> **Trajectory compression differs from context compression: the latter serves inference quality, the former serves training signal quality.** This means when judging "what to compress, what to keep", the standard is not "is this important for the current task", but "is this important for model learning"

**Three-Priority Model of Training Signals:**

| Priority | Content | Reason |
|---|---|---|
| **P0 Must Retain** | system prompt + first human + first gpt + first tool | "Anchors" for behavioral pattern learning, determining the initial direction of strategy |
| **P1 Must Retain** | last 4 turns | Final state and conclusion directly affect reward, tail contains clearest causal signals |
| **P2 Can Compress** | All middle turns | As long as summary can retain key decision paths, details of intermediate process can be compressed |

**Why is the summary strategy "descriptive summary" not "reasoning process extraction"?**

- Summary prompt requires: describe what the assistant **did** (tool calls, searches, file operations), not what it **thought**
- Reason: In training data, the model's tool call sequence is the behavior to be learned, not the reasoning chain
- This contrasts with `reasoning_per_turn`: reasoning content is separately extracted for display (wandb) but not injected into training data

**Meta-value of compression metrics: Signal itself is data**

- `TrajectoryMetrics` records: raw tokens, compressed tokens, compression ratio, API call count, error count
- These metrics are not just "logs" but **implicit feedback signals for compression quality**
- Example: `still_over_limit: true` means compression wasn't aggressive enough, next time need to adjust `target_max_tokens`
- `compression_ratio` distribution (min/max/median) is the basis for judging whether compression strategy is stable

### 3.5.3 Atropos RL Environment Integration

**File**: `environments/hermes_base_env.py` / `environments/agent_loop.py` / `tools/rl_training_tool.py`

Integration with Tinker-Atropos RL framework:

```
# Two run modes:
# Phase 1: OpenAI compatible server
# Phase 2: vLLM ManagedServer (GPU-accelerated inference)

# Reward function design:
class ToolContext:
    """Provides tool access for each rollout, used for custom reward computation"""
    get_tool_calls()      # Get tool call sequence
    get_rewards()         # Compute reward signals
    get_trajectory()      # Get complete trajectory
```

**Training Environments:**

| Environment | Purpose |
|---|---|
| `HermesSweEnv` | SWE-bench style software engineering tasks |
| `TerminalBench2EvalEnv` | Terminal benchmark evaluation |
| `TerminalTestEnv` | Stack verification tests |

---

# 4. Auxiliary Evolution Subsystems

## 4.1 User Modeling (Honcho)

**File**: `tools/honcho_tools.py` / `honcho_integration/session.py`

> **Important**
>
> **Core Design Insight: Honcho's "Dialectic Understanding" is essentially distilling user feedback signals into structured priors**
>
> Honcho doesn't store what users "said", but what they "understood" — this is a distillation process from raw signals to abstract representations (or alternatively, a process of **implicit knowledge mining** from interaction history). This design solves a fundamental problem: **Users won't proactively tell you who they are; they'll only tell you what they want.**

Honcho is Nous Research's AI-native user modeling system (specializing in storing knowledge "about users" such as preferences, styles, habits), providing **dialectic user understanding** beyond simple key-value storage:

| Tool | Function |
|---|---|
| `honcho_profile` | Retrieve/update user refined profile (peer card) |
| `honcho_search` | Semantic search of stored user context |
| `honcho_context` | LLM-driven dialectic Q&A (understand user intent) |
| `honcho_conclude` | Write conclusions back to Honcho long-term memory |

> **Note**
>
> **Honcho's storage medium: External service (HTTP API)**

Unlike SQLite and file systems, Honcho's data is stored on Nous Research's Honcho server, with local maintenance of only a message cache ( `honcho_integration/session.py` ), synchronized with the service via HTTP API. This design **makes user profiles naturally cross-device and cross-session shared** — switching to another machine still reads the same profile.

The cost is: dependency on network availability, server data cannot be locally directly edited, and consistency between it and local MEMORY.md / USER.md needs manual synchronization via `honcho_conclude` .

**User Modeling Flow:**

```
Each session interaction
    ↓
Collect key information (preferences, habits, feedback)
    ↓
honcho_conclude: Write back to Honcho
    ↓
Subsequent sessions honcho_profile: Retrieve user profile
    ↓
honcho_search: Semantic search historical context
    ↓
Agent adjusts response strategy based on user profile
```

## 4.2 Multi-Instance Profile Isolation

**File**: `hermes_cli/profiles.py`

Hermes Agent supports multi-instance Profiles with completely isolated runtime environments:

```
~/.hermes/
├── config.yaml              # default profile
├── .env
├── memories/
├── skills/
├── sessions.db
└── profiles/
    ├── work/                 # work scenario profile
    │   ├── config.yaml
    │   ├── .env
    │   ├── memories/
    │   ├── skills/
    │   └── sessions.db
    ├── research/             # research scenario profile
    │   └── ...
    └── personal/
        └── ...
```

**Isolated Contents:**

- Configuration files (config.yaml)
- API keys (.env)
- Memory files (memories/)
- Skill collections (skills/)
- Session history (sessions.db)

- Gateway configuration (gateway/)
- Scheduled tasks (cron/)

> **Note**
>
> Honcho's design in Hermes is user-oriented (people), not profile-oriented:
>
> - Profile is "work scenario isolation"
> - Honcho is "cognitive sharing of the same person"
>
> So if you said a lot about work preferences using work Profile, after switching to personal Profile, Honcho still remembers — completely shared across Profiles

### 4.3 MCP Dynamic Tool Discovery

**File**: `tools/mcp_tool.py`

MCP (Model Context Protocol) support enables the Agent to **dynamically extend tool capabilities**:

```
# MCP tool registration flow
mcp.discover_servers()     # Discover available MCP servers
    ↓
mcp.list_tools()          # Get tool list
    ↓
mcp.call_tool(name, args) # Invoke tool
    ↓
notifications/tools/list_changed  # Listen for tool changes
    ↓
Dynamically update Agent toolset
```

---

## 5. Tool Layer Self-Evolution Support

### Tool Registry

**File**: `tools/registry.py`

A unified tool registry supporting tool discovery, registration, permission control, and distribution:

```
# ── Tool Registry Registry ──────────────────────────────────────
# Each tool module calls register() on import, declaring itself to the global registry
# model_tools.py only queries the registry, no longer maintains its own tool list
# ──────────────────────────────────────────────────────────────────

registry.register(
    name="skill_manage",          # Unique identifier for the tool, dispatch looks up by name
    toolset="skills",             # Toolset this tool belongs to, used for batch enable/disable by
toolset

    # ── OpenAI tool_calls format schema ──────────────────────
    # Tells LLM: what this tool is called, what it does, what parameters it accepts
    schema={
        "name": "skill_manage",   # Must match name field within schema
        "description": "Create, edit, or patch skills...",
        "parameters": {
            "type": "object",
```

```
            "properties": {
                "action": {
                    "type": "string",
                    "enum": ["create", "edit", "patch", "delete"]
                    # action determines operation type:
                    #   create   — create new skill from successful experience
                    #   edit     — completely rewrite skill content
                    #   patch    — quick patch (for specific errors/omissions)
                    #   delete   — delete obsolete skills
                },
                ...
            }
        }
    },

    # ── Actual execution logic for the tool ──────────────────────────────
    # The real work function, called when registry.dispatch(name)
    # Can be a regular function or async function (marked by is_async)
    handler=lambda args, **kw: skill_manager_tool(...),

    # ── Availability check function ──────────────────────────────────────
    # check_fn() is called before dispatch, tool is only exposed to LLM if it returns True
    # Returns False = tool is invisible to LLM (not an error)
    # check_fn can check: environment variables, dependencies installed, API key configured, etc.
    # skill_manage requires HERMES_SKILLS_DIR to exist, otherwise skills cannot be created
    check_fn=check_requirements,

    # ── List of required environment variables ──────────────────────────
    # Not a hard constraint (check_fn is), but metadata
    # Used for hints/documentation: informs what environment variables this tool depends on
    # MCP dynamic discovery will check these, ensuring tool can run properly
    requires_env=["HERMES_SKILLS_DIR"],
)
```

## Tool Classification (Toolsets)

**File**: `toolsets.py`

```
TOOLSETS = {
    "web":        ["web_tools"],
    "search":     ["web_tools"],
    "vision":     ["vision_tool"],
    "image_gen":  ["image_generation_tool"],
    "terminal":   ["terminal_tool", "environments"],
    "skills":     ["skills_tool", "skill_manager_tool"],
    "browser":    ["browser_tool"],
    "cronjob":    ["cron_tool"],
    "messaging":  ["messaging_tools"],
    "rl":         ["rl_training_tool", "trajectory_compressor"],
    "file":       ["file_tools"],
    "tts":        ["tts_tool"],
    "todo":       ["todo_tool"],
    "memory":     ["memory_tool"],
    "session_search": ["session_search_tool"],
    "clarify":    ["clarification_tool"],
```

```
    "code_execution": ["code_execution_tool"],
    "delegation": ["delegate_tool"],
    "honcho":     ["honcho_tools"],
    "homeassistant": ["homeassistant_tool"],
}
```

---

# 6. Trajectory Engineering: Data-Driven Evolution

## Complete Trajectory Lifecycle

```
Step 1: Session Execution
        run_agent.py / batch_runner.py
             ↓
Step 2: Trajectory Collection
        agent/trajectory.py → JSONL file
        (Complete interaction history for each session)
             ↓
Step 3: Batch Processing
        batch_runner.py → ShareGPT format
        (Process multiple prompts in parallel, count tool usage)
             ↓
Step 4: Trajectory Compression
        trajectory_compressor.py
        (Protect training signals, compress intermediate history)
             ↓
Step 5: RL Training
        rl_cli.py + environments/ + Atropos
        (Generate trainable RL data or SFT data)
             ↓
Step 6: Model Update
        New model replaces old model
             ↓
Step 7: Closed-Loop Verification
        New model performs better on tasks
        → Produces better trajectories → Better data → ...
```

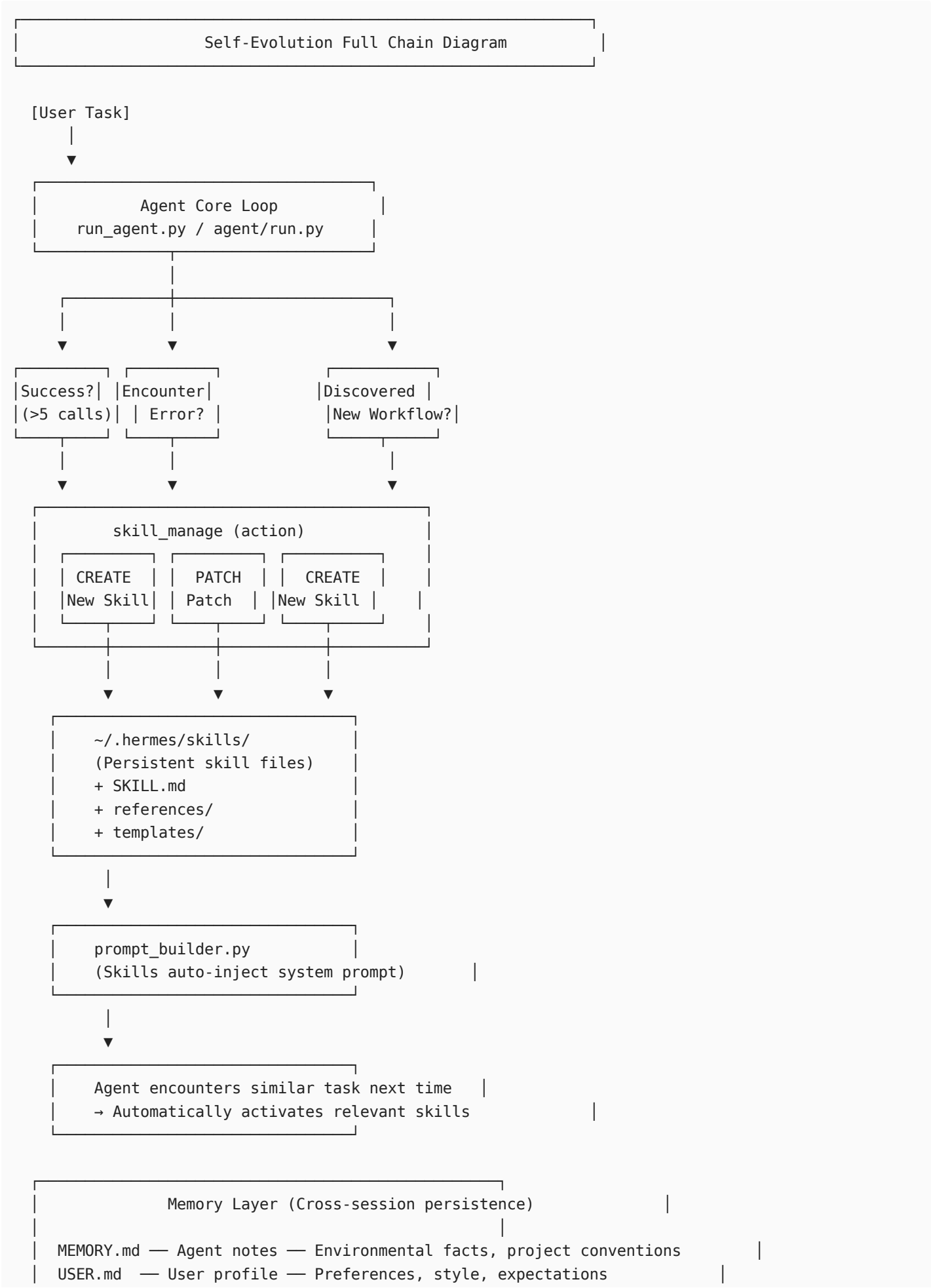
## Batch Runner

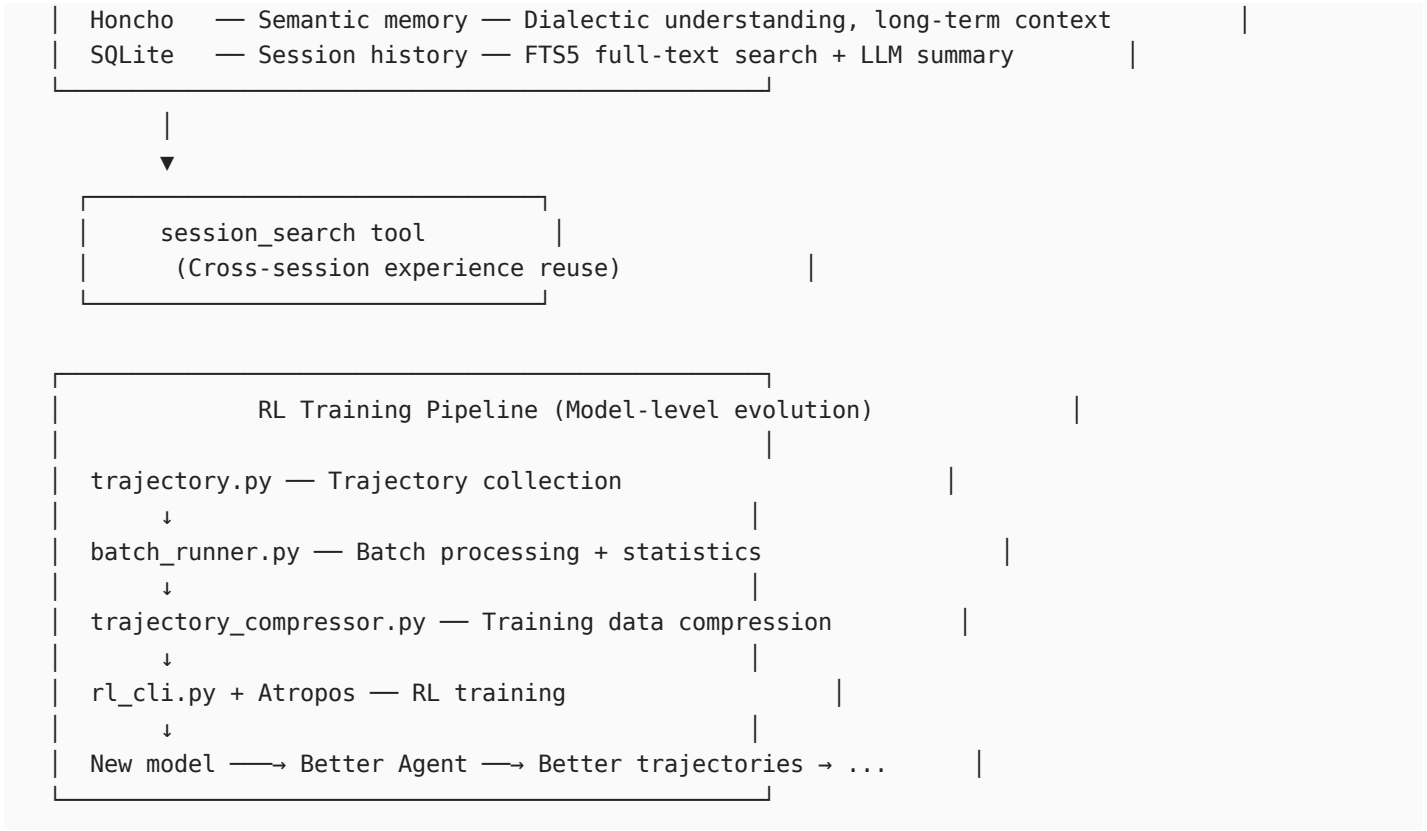
**File**: `batch_runner.py`

```
# Batch run mode
batch_runner.run(
    prompts_file="prompts.jsonl",
    num_parallel=8,       # Parallel count
    model="claude-sonnet-4-6",
    save_trajectories=True,
    tool_stats=True,      # Tool usage statistics
    checkpoint_dir="./checkpoints",
)
```

---

# 7. Evolution Mechanism Relationship Diagram


Self-Evolution Full Chain Diagram
[User Task]
Agent Core Loop
run_agent.py / agent/run.py
Success?
(>5 calls)
Encounter
Error?
Discovered
New Workflow?
skill_manage (action)
CREATE
New Skill
PATCH
Patch
CREATE
New Skill
~/.hermes/skills/
(Persistent skill files)
+ SKILL.md
+ references/
+ templates/
prompt_builder.py
(Skills auto-inject system prompt)
Agent encounters similar task next time
→ Automatically activates relevant skills
Memory Layer (Cross-session persistence)
MEMORY.md —— Agent notes —— Environmental facts, project conventions
USER.md —— User profile —— Preferences, style, expectations




---

# 8. Technical Implementation Details

## 8.1 Skill Activation Mechanism

Skills are injected into the system prompt through `SKILLS_GUIDANCE` in `prompt_builder.py`:

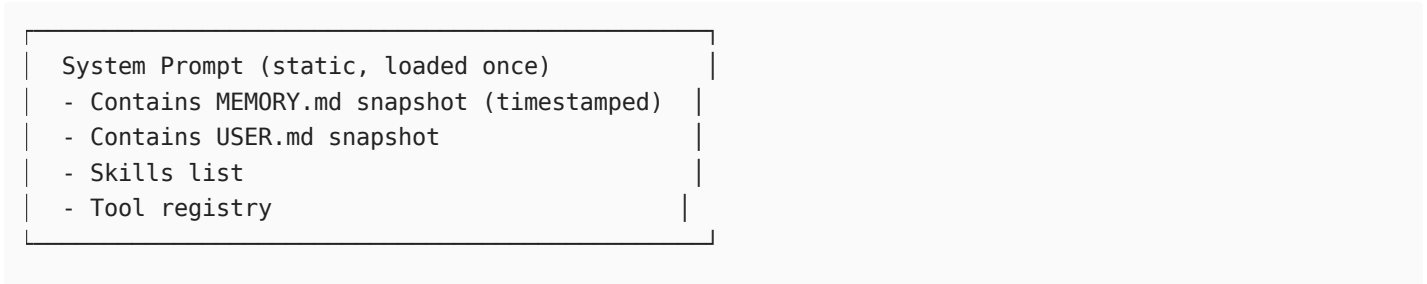
```
SKILLS_GUIDANCE = """
After completing a complex task (5+ tool calls), fixing a tricky error,
or discovering a non-trivial workflow, save the approach as a
skill with skill_manage so you can reuse it next time.

When using a skill and finding it outdated, incomplete, or wrong,
patch it immediately with skill_manage(action='patch') - don't wait to be asked.
Skills that aren't maintained become liabilities.
"""
```

This means skill activation is implemented through **LLM's reasoning ability** rather than hardcoded rules — the Agent autonomously decides whether to create/use/patch skills based on the current task context.

## 8.2 Memory Injection Strategy

Memory is injected through the "frozen snapshot" mode in `memory_tool.py`:



```
                    ↓
┌──────────────────────────────────────────────────┐
│  Tool Response (dynamic, each call)              │
│  - memory tool returns latest memory content     │
│  - session_search returns latest search results  │
│  - honcho_profile returns latest user profile    │
└──────────────────────────────────────────────────┘
```

## 8.3 Security Mechanisms

| Security Measure | Implementation Location | Description |
|---|---|---|
| Injection attack scanning | `memory_tool.py` | Scan injection patterns in `MEMORY.md` / `USER.md` |
| Skill security scanning | `tools/skills_hub.py` | Validate skill content before installation |
| Skill isolation | `~/.hermes/skills/` | Skills in separate directories, avoid mutual interference |
| Skill isolation (Hub) | quarantine mechanism | Untrusted skills enter quarantine zone |

## 8.4 Platform Adaptation Layer

**File**: `gateway/platforms/`

```
gateway/
├── run.py              # Unified gateway entry
└── platforms/
    ├── telegram.py     # Telegram adapter
    ├── discord.py      # Discord adapter
    ├── slack.py        # Slack adapter
    ├── whatsapp.py     # WhatsApp adapter
    └── signal.py       # Signal adapter
```

Sessions from each messaging platform are uniformly stored in SQLite, session retrieval can be cross-platform.

---

# 9. Summary and Evaluation

## 9.1 Core Design Philosophy

Hermes Agent's self-evolution mechanism follows the **"Safe Evolution" principle** — not modifying the Agent's own code, but achieving capability accumulation and reuse through a three-layer persistent knowledge system:

```
Experience → Skills (persistent knowledge) → Cross-session reuse → Better performance
  ↓
Trajectory → Training data → Model update → Stronger capabilities
```

## 9.2 Evolution Mechanism Classification

| Evolution Layer | Specific Mechanism | Reversibility | Scope of Effect |
|---|---|---|---|
| **Skill Layer** | Create/patch skills | High (can edit/delete) | Specific task types |
| **Memory Layer** | MEMORY.md / USER.md | High (can edit) | Agent behavior style |
| **Session Retrieval Layer** | FTS5 search + LLM summary | Medium (covers old sessions) | Context understanding |
| **User Modeling Layer** | Honcho profiles | Medium (continuously updated) | Personalized interaction |

| Evolution Layer | Specific Mechanism | Reversibility | Scope of Effect |
|---|---|---|---|
| **Trajectory/RL Layer** | Trajectory compression + RL training | Low (model update) | General capabilities |

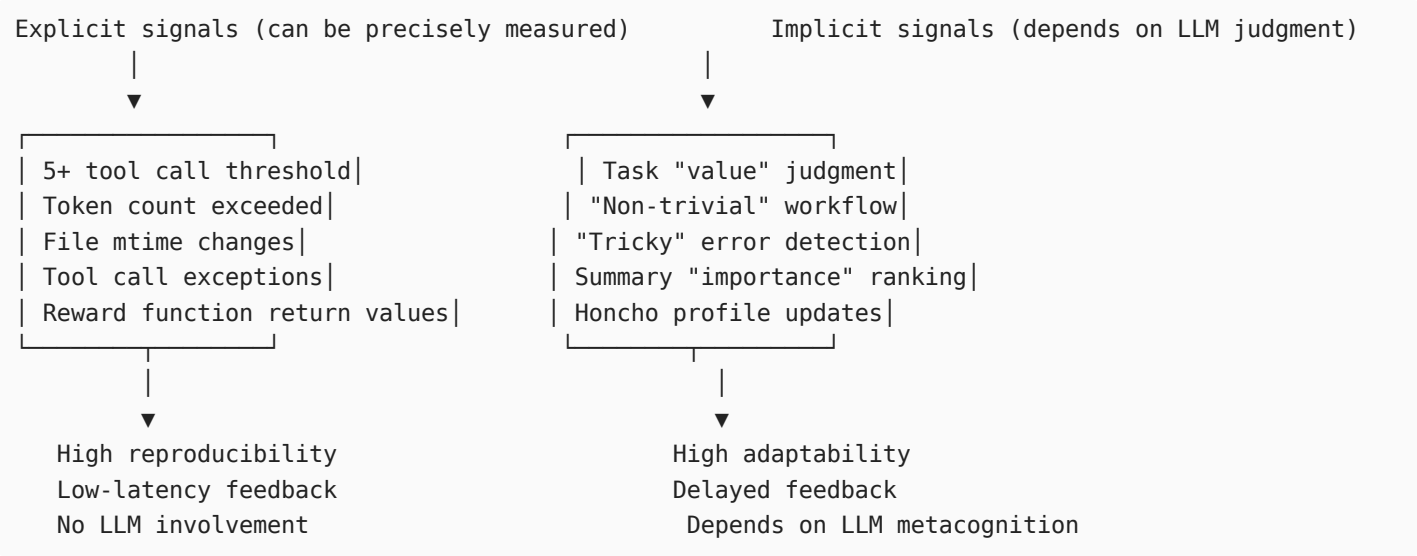
## 9.3 Innovation Points

1. **Skills as first-class citizens**: Skills are not simple prompt fragments, but complete knowledge modules with versions, platform adaptation, file structure, and reference sub-documents
2. **Frozen snapshot mode**: System prompt is stable but memory tool responses are dynamic, solving the contradiction between consistency and freshness
3. **FTS5 + LLM dual-layer retrieval**: Full-text index guarantees speed, LLM summary guarantees relevance
4. **Dialectic user modeling**: Honcho is not a key-value store, but a system that "dialectically" understands user intent
5. **Trajectory compression protects training signals**: Not simple truncation, but semantically meaningful compression that protects key training signals

## 9.4 Comprehensive Analysis of Feedback Signal System

> **Important**
>
> **This is the core framework for understanding Hermes Agent's self-evolution mechanism: Feedback signals, not code structure, are what determine the direction and speed of evolution.**

**"Explicit-Implicit" Spectrum of Feedback Signals:**




**Five Feedback Signal Types and Their Distribution in the System:**

| Signal Type | Typical Examples | Feedback Latency | Manifestation in Hermes |
|---|---|---|---|
| **Structural** | Tool call count, token count | Extremely low (millisecond level) | Skill creation threshold, compression trigger |
| **Result** | Task success/failure, tool errors | Low (second level) | RL reward computation, tool error tracking |
| **Social** | User corrections, user satisfaction | Medium (minute to day) | Honcho conclude, USER.md updates |

| Signal Type | Typical Examples | Feedback Latency | Manifestation in Hermes |
|---|---|---|---|
| **Semantic** | LLM self-assessment, pattern recognition | Medium (depends on LLM speed) | Skill creation decisions, summary quality |
| **Environmental** | mtime changes, file existence | Low (filesystem level) | Skill cache invalidation |

**The Metacognitive Dilemma of Implicit Signals: Who verifies the verifier?**

- Skill creation depends on Agent's judgment "is this worth remembering" — but the Agent's judgment itself is limited by its cognitive ability
- If the Agent **incorrectly underestimates** the value of an experience, it won't create a skill → evolution stagnates
- If the Agent **incorrectly overestimates** the value of an experience, it creates noisy skills → skill library degrades
- This is a **self-referential feedback loop**: signal quality depends on the system itself, and the system itself is affected by signals

**Most Elegant Design: Tool Call Pair Integrity ( `_sanitize_tool_pairs` )**

- This is a kind of **implicit constraint feedback**: not directly saying "this action is wrong", but saying "the return value of this action is missing"
- Essentially using **structural integrity** rather than **content correctness** to detect problems
- Contrast with RL: This is equivalent to "valid action space constraints" — Agent can make any mistake but cannot break the MDP state transition contract

## 9.5 Limitations

1. **No code-level self-modification**: Evolution is limited to prompts, memory, and skills, not modifying underlying logic
2. **Skill quality depends on Agent judgment**: If the Agent misjudges "when to create skills", it may produce noisy skills
3. **RL training pipeline requires external Atropos**: Only provides data, does not include complete RL training implementation
4. **No explicit evolution evaluation mechanism**: No quantitative metrics to measure the degree or speed of "evolution"
5. **Skills not shared across Profiles**: Different Profiles have completely isolated skills, cannot inherit

---

# Reference File Index

| File | Purpose |
|---|---|
| `tools/skill_manager_tool.py` | Skill CRUD core implementation |
| `tools/memory_tool.py` | File memory system |
| `tools/session_search_tool.py` | Cross-session FTS5 retrieval |
| `tools/honcho_tools.py` | Honcho user modeling interface |
| `agent/prompt_builder.py` | System prompt construction and skill injection |
| `agent/context_compressor.py` | Context window compression |
| `agent/trajectory.py` | Trajectory collection |
| `trajectory_compressor.py` | Training data compression |
| `batch_runner.py` | Batch trajectory collection |
| `rl_cli.py` | RL training CLI |
| `environments/hermes_base_env.py` | Atropos RL environment |

| File | Purpose |
| --- | --- |
| `hermes_state.py` | SQLite session storage |
| `hermes_cli/profiles.py` | Multi-Profile isolation |
| `toolsets.py` | Toolset definitions |
| `tools/registry.py` | Tool registry |